
pyconcurrent API Reference

Release 4.0.1

Gene C

Oct 08, 2026

API REFERENCE GUIDE:

1	pyconcurrent	1
1.1	Overview	1
1.2	Signed Source	1
1.3	pyconcurrent module	1
2	Examples	3
2.1	Example 1a: Executable with Asyncio	3
2.2	Example 1b: Executable with Multiprocessing	4
2.3	Example 2: Function with Asnycio	4
2.4	Example 3: Executable with run_prog	5
3	API Reference	7
3.1	class cidrtools	7
4	Appendix	11
4.1	Installation	11
4.2	Dependencies	11
4.3	License	12
	Python Module Index	13

PYCONCURRENT

1.1 Overview

pyconcurrent is a python class that provides a simple way to do concurrent processing. It supports both asyncio and multiprocessing. The tasks to be run concurrently can either be an executable which is run as a subprocess or a python function to be called.

1.1.1 Key features

- Provides two classes to do the work: *ProcRunAsyncio* and *ProcRunMp*
- Results are provided by the *results* attribute in each class. This is a list of *ProcResults*; one per run.
- Documentation includes the API reference.
- pytest classes validate that all functionality works as it should.

1.1.2 Documentation

The manual provides detailed information and is available in both HTML and PDF formats. Both are installed under `/usr/share/pyconcurrent/docs`.

The manual is also available at: [readthedocs](#).

1.2 Signed Source

All git tags are signed with arch@sapience.com key which is available via WKD or download from <https://www.sapience.com/tech>. Add the key to your package builder gpg keyring. The key is included in the Arch package and the `source=` line with `?signed` at the end can be used to verify the git tag. You can also manually verify the signature using `git tag -v <tag-name>`

1.3 pyconcurrent module

Please see the API reference manual for details. The manual, available in both PDF and html, has illustrative examples using:

- *ProcRunAsyncio* class
- *ProcRunMp* class
- `run_prog`

EXAMPLES

Examples running an executable synchronously, using multiprocessing and using asyncio. These use:

- ProcRunAsyncio class
- ProcRunMp class
- run_prog

along with ProcResult class.

2.1 Example 1a: Executable with Asyncio

This example uses asyncio and subprocesses to call an executable. *tasks* must be a list of (*key*, *arg*) pairs, 1 per task.

key is a unique identifier, used by caller, one per task. *arg* is an additional argument for each task; typically *arg* provides for whatever work that task is responsible for. Each *result* returned contains both the *key* and the *arg* used by that task, information about the success of the task as well as any outputs produced by the task. See *ProcResult* class for more detail.

This example has 5 tasks to be run concurrently, at most 4 at a time. The results are available in the *proc_run.result*, which is a list of *ProcResult* items; one per task. Since the result order is not pre-defined, each task is identifiable by its *key* available in the : *result.key*.

Listing 1: Example 1a

```
#!/usr/bin/python
"""
Example 1
"""
import asyncio
from pyconcurrent import ProcRunAsyncio

async def main():
    """
    pargs can have additional arguments.
    """
    pargs = ['/usr/bin/sleep']
    tasks = [(1, 1), (2, 7), (3, 2), (4, 2), (5, 1)]

    proc_run = ProcRunAsyncio(pargs, tasks, num_workers=4, timeout=30)
    await proc_run.run_all()
```

(continues on next page)

(continued from previous page)

```

proc_run.print_results()

if __name__ == '__main__':
    asyncio.run(main())

```

2.2 Example 1b: Executable with Multiprocessing

To switch to *multiprocessing* simply replace *ProcRunAsyncio* with *ProcRunMp*, and drop *await* since MP is not *async*. i.e.

Listing 2: Example 1b

```

#!/usr/bin/python
"""
Example 1b
"""
from pyconcurrent import ProcRunMp

def main():
    """
    Multiprocessing
    """
    pargs = ['/usr/bin/sleep']
    tasks = [(1, 1), (2, 7), (3, 2), (4, 2), (5, 1)]

    proc_run = ProcRunMp(pargs, tasks, num_workers=4, timeout=30)
    proc_run.run_all()
    proc_run.print_results()

if __name__ == '__main__':
    main()

```

2.3 Example 2: Function with Asnycio

The next example uses a caller supplied function together with *asyncio*. As in the first example, there are 5 tasks to do and the number of workers is 4, so that 4 tasks are permitted to be run simultaneously.

Listing 3: Example 2

```

#!/usr/bin/python
"""
Example 2
"""
import asyncio
from typing import Any
from pyconcurrent import ProcRunAsyncio

async def test_func_async(key, args) -> tuple[bool, dict[str, Any]]:
    """
    return 2-tuple (success, result).
    """
    success = True
    nap_time = args[-1]      # pull off the last argument

```

(continues on next page)

(continued from previous page)

```

    await asyncio.sleep(nap_time)
    answer: dict[str, Any] = {
        'key': key,
        'args': args,
        'success': success,
        'result': 'test_func done',
    }
    return (success, answer)

async def main():
    """
    pargs can have additional arguments.
    """
    pargs = [test_func_async]
    tasks = [(1, 1), (2, 7), (3, 2), (4, 2), (5, 1)]

    proc_run = ProcRunAsyncio(pargs, tasks, num_workers=4, timeout=30)
    await proc_run.run_all()
    proc_run.print_results()

if __name__ == '__main__':
    asyncio.run(main())

```

For equivalent multiprocessor version for this one, same as above, simply replace *ProcRunAsyncio* with *ProcRunMp* and drop any references to **async/await**.

The caller supplied function here, *test_func_async()*, must return a 2-tuple of (*success:bool*, *answer:Any*) where success should be *True* if function succeeded.

The function may optionally raise a *RuntimeError* exception, but typically setting *success* is sufficient. If you are using exceptions then please use this one.

2.4 Example 3: Executable with run_prog

This example illustrates a non-concurrent external program being executed.

Listing 4: Example 3

```

#!/usr/bin/python
"""
Non-Concurrent
"""
from pyconcurrent import run_prog

def main():
    """
    Run external program with arguments
    """
    pargs_good = ['/usr/bin/sleep', '1']
    pargs_bad = ['/usr/bin/false']

    for pargs in [pargs_good] + [pargs_bad]:
        print(f'Testing: {pargs}:')
        (ret, stdout, stderr) = run_prog(pargs)

        if ret == 0:

```

(continues on next page)

(continued from previous page)

```
        print('\tAll well')
        print(stdout)
    else:
        print('\tFailed')
        print(stderr)

if __name__ == '__main__':
    main()
```

API REFERENCE

This section contains the API documentation for the `pyconcurrent` package.

3.1 class `cidrtools`

Public Methods. `pyconcurrent`.

class `pyconcurrent.ProcResult`(*key*, *arg*)

Bases: `object`

Result of running one of the concurrent processes.

Args:

key (Any):

Caller provided unique identifier.

arg (Any):

The additional argument used for this run.

Attributes:

time_start (float):

Unix time in seconds.

time_run (float):

Seconds taken for this item to complete.

success (bool):

True if completed successfully.

timeout (bool):

True if failed to complete in less than timeout restriction.

key (Any):

The caller provided unique identifier.

arg (Any):

The called provided argument for this run.

returncode (int):

Return value of subprocess. Typically 0 for success.

stdout (str):

Returned stdout of subprocess.

stderr (str):

Returned stderr of subprocess.

answer (Any):

Return provided by the function.

print()

Testing: simple print attributes.

```
class pyconcurrent.ProcRun(pargs: list[Any], tasks_todo: list[tuple[Any, Any]], mp_type:
                             MPType, num_workers: int = 4, timeout: int | float = 0, verb: bool =
                             False)
```

Bases: object

Parallelization via asyncio / multiprocessing.

Base Class used by ProcRunMP / ProcRunAsyncio.

print_results()

Test tool : prints each result using the ProcResul::print().

```
class pyconcurrent.ProcRunAsyncio(pargs: list[Any], tasks_todo: list[tuple[Any, Any]],
                                   num_workers: int = 4, timeout: int | float = 0, verb: bool
                                   = False)
```

Bases: [ProcRun](#)

Run concurrent processes using asyncio.

Asyncio concurrent process runs. Supports program to be run as a subprocess or a function to be called. The result of each run is returned as in ProcResult class instance.

Inherits from base class ProcRun:

async run_all()

Start running all the provided commands/functions concurrently.

Awaitable, so caller is responsible for calling asyncio.run(). See run_all_start_asyncio() for non-awaitable version.

```
class pyconcurrent.ProcRunMp(pargs: list[Any], tasks_todo: list[tuple[Any, Any]],
                              num_workers: int = 4, timeout: int | float = 0, verb: bool =
                              False)
```

Bases: [ProcRun](#)

Run concurrent processes using multiprocessing.

Inherits from base class ProcRun with same calling convention as ProcRunAsyncio.

Note: func cannot be async func() - conflicts with mp starmap using async

run_all()

Do the work

```
pyconcurrent.run_prog(pargs: list[str], input_str: str | None = None, stdout: int = -1, stderr: int =  
-1, env: dict[str, str] | None = None, test: bool = False, verb: bool = False)  
→ tuple[int, str, str]
```

Run external program using subprocess.

Take care to handle large outputs (default buffer size is 8k). This avoids possible hangs should IO buffer fill up.

non-blocking IO together with select() loop provides a robust methodology.

Args:

pargs (list[str]):

The command + arguments to be run in standard list format. e.g. ['/usr/bin/sleep', '22'].

input_str (str | None):

Optional input to be fed to subprocess stdin. Defaults to None.

stdout (int):

Subprocess stdout. Defaults to subprocess.PIPE

stderr (int):

Subprocess stderr. Defaults to subprocess.PIPE

env (None | dict[str, str]):

Optional to specify environment for subprocess to use. If not set, inherits from calling process as usual.

test (bool):

Flag - if true dont actually run anything.

verb (bool):

Flag - only used with test == True - prints pargs.

Returns:

tuple[retc: int, stdout: str, stderr: str]:

retc is 0 when all is well. stdout is what the subprocess returns on it's stdout and stderr is what it's stderr return.

Note that any input string is written in it's entirety in one shot to the subprocess. This should not be a problem.

4.1 Installation

Available on

- [Github](#)
- [Archlinux AUR](#)

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. All git tags are signed with arch@sapience.com key which is available via WKD or download from <https://www.sapience.com/tech>. Add the key to your package builder gpg keyring. The key is included in the Arch package and the source= line with *?signed* at the end can be used to verify the git tag. You can also manually verify the signature:

```
git tag -v <tag-name>
```

To build manually, clone the repo and

```
./scripts/do-build  
./scripts/do-install <destination-directory>
```

4.2 Dependencies

Run Time :

- python (3.14 or later)
- dateutil

Building Package :

- git
- meson
- meson-python
- rsync
- pytest
- pytest-asyncio

4.3 License

Created by Gene C. and licensed under the terms of the GPL-2.0-or-later license.

- SPDX-License-Identifier: GPL-2.0-or-later
- SPDX-FileCopyrightText: © 2025-present Gene C <arch@sapience.com>

PYTHON MODULE INDEX

p

pyconcurrent, [7](#)